

Open Problems and Current Solutions

Neural Network–Accelerated OT Resampling

April 1, 2026

Reading Guide

Each problem is tagged: **[RESOLVED]** = we have a concrete solution; **[PARTIAL]** = we have a direction but gaps remain; **[OPEN]** = unsolved or undiscussed. Problems are ordered by how early they block progress.

1. P1: Sinkhorn is infeasible at production scale **[RESOLVED]**

Problem

Production requires $N = 10^6$ particles. Sinkhorn costs $O(LN^2)$ per resampling step. The cost matrix $C \in \mathbb{R}^{N \times N}$ has 10^{12} entries (~ 4 TB at float32). Cannot be stored, let alone iterated on.

Consequence

The transport plan $\pi \in \mathbb{R}^{N \times N}$ also cannot be stored. Any method that outputs or manipulates π as a dense matrix is ruled out. This eliminates the “Step 1” direct map formulation ($f_\theta : (\mathbf{w}^0, \mathbf{w}^1, \mathbf{X}) \rightarrow \pi$) as a production solution, though it remains useful as a small- N proof of concept.

Solution

Work with the transport *map* $T : \mathbb{R}^d \rightarrow \mathbb{R}^d$ instead of the transport *plan*. The map is evaluated pointwise: $\tilde{\mathbf{x}}_i = T(\mathbf{x}_i)$, cost $O(1)$ per particle, $O(N)$ total. This requires the semi-discrete or fully continuous formulation (a neural operator that outputs a function, not a matrix). At $N = 10^6$ this is not a theoretical nicety—it is the only feasible approach.

2. P2: The transport map is not differentiable **[RESOLVED]**

Problem

The Monge map $T(\mathbf{x}) = \mathbf{y}_j$ for $\mathbf{x} \in L_j$ (Laguerre cell) is piecewise constant. Discontinuous at cell boundaries. Backpropagation through hard assignments fails.

Solution

Entropy regularization. Replace hard Laguerre cells with soft assignments:

$$\pi_j(\mathbf{x}) = \rho_0(\mathbf{x}) \cdot \frac{\exp((\psi_j - c(\mathbf{x}, \mathbf{y}_j))/\varepsilon)}{\sum_k \exp((\psi_k - c(\mathbf{x}, \mathbf{y}_k))/\varepsilon)}$$

Resampled position $\tilde{\mathbf{x}}_i = \sum_j \pi_j(\mathbf{x}_i) \mathbf{y}_j$ is smooth in all arguments. The full pipeline (KDE $\rightarrow$ neural operator $\rightarrow$ soft assignment $\rightarrow$ resample) is end-to-end differentiable.

3. P3: Neural operator gradients are biased [RESOLVED]

Problem

The neural operator outputs $\hat{\psi} \approx \psi^*$. Backpropagating through the network gives $\partial\hat{\psi}/\partial\mathbf{w}$, not the true implicit-function gradient $\partial\psi^*/\partial\mathbf{w}$. With approximation error, these differ, biasing the parameter gradients in the outer inference loop.

Solution

Use the neural operator output as a warm start for 2–3 Newton iterations on the semi-discrete dual. Differentiate through the Newton steps via implicit differentiation at the fixed point. This gives correct gradients at convergence while keeping iteration count low. Worst case: if the warm start is poor, fall back to more iterations (see P8).

4. P4: Training data validity under parameter inference [RESOLVED]

Problem

When model parameters are unknown (inference setting), the filter runs with wrong parameters. Are the transport plans computed during inference still valid training data?

Solution

Yes. Each Sinkhorn solution is correct for its specific input $(\mathbf{w}^0, \mathbf{w}^1, \mathbf{X})$. The OT solver does not depend on whether the weights came from a well-specified model. Every (input, ψ^*) pair is a valid training example.

What changes is the *distribution* of inputs. Early in inference (poor parameters): degenerate weights, concentrated particles. Late (good parameters): balanced weights, dispersed particles. The DAgger bootstrapping loop naturally collects data from all regimes. The inference setting actually provides *more* input diversity for free.

5. P5: Representing ρ_0 at scale [PARTIAL]

Problem

The semi-discrete formulation requires a continuous source density ρ_0 . The natural choice is KDE: $\rho_0(\mathbf{x}) = \sum_i w_i^0 K_h(\mathbf{x} - \mathbf{x}_i)$. But evaluating this at a query point costs $O(N)$ (sum over all N particles). At $N = 10^6$, this is expensive—and the branch net of DeepONet needs to encode ρ_0 as a finite-dimensional vector.

Partial solution

Compress ρ_0 into a fixed-dimensional representation before feeding it to the branch net. Options:

- **Random Fourier features:** project the empirical measure onto m random Fourier basis functions. Cost $O(Nm)$ to compute the m -dimensional embedding. This is a kernel mean embedding of the distribution.
- **Grid discretization** (at $d = 5$): evaluate ρ_0 on a coarse grid, e.g., $20^5 \approx 3.2\text{M}$ points. Feasible for the proof of concept but scales as M^d .
- **Spatial tree + fast KDE:** kd-tree or ball-tree for approximate KDE evaluation in $O(\log N)$

per query.

Open questions

- What compression dimension m is needed? Depends on the intrinsic complexity of the distributions encountered during filtering. Unknown without experiments.
- Does the compression error compound with the neural operator error and the entropic regularization error in a controllable way?
- At $d = 50$, grid discretization is impossible. Random features or learned embeddings are the only options. Their quality at $d = 50$ for this specific problem is untested.

6. P6: Soft assignment cost at $N = 10^6$ [OPEN]

Problem

Even with a learned map that outputs $\psi^* \in \mathbb{R}^N$, the soft resampled position $\tilde{\mathbf{x}}_i = \sum_j \pi_j(\mathbf{x}_i) \mathbf{y}_j$ sums over all N target atoms. This is $O(N)$ per source particle, $O(N^2)$ total. At $N = 10^6$, this reintroduces the quadratic bottleneck.

Possible directions

- **Truncated softmax:** for each source particle $\mathbf{x}_i$, only sum over the k nearest target atoms $\mathbf{y}_j$ (using a spatial index). The softmax decays exponentially with distance when ε is small, so distant targets contribute negligibly. Cost $O(Nk)$ with $k \ll N$.
- **Hard map with straight-through estimator:** use the hard assignment $T(\mathbf{x}_i) = \mathbf{y}_{j^*}$ in the forward pass but the soft gradient in the backward pass. Avoids the sum entirely in the forward pass. Gradient quality depends on how well the straight-through approximation works for this problem.
- **Continuous map output:** instead of outputting dual variables ψ and reconstructing via softmax over discrete targets, have the neural operator directly output $T(\mathbf{x}_i) \in \mathbb{R}^d$. The output is a point in continuous space, not a weighted sum over atoms. This is a Monge map, not a Kantorovich plan. Differentiable if the network is differentiable. But loses the marginal constraint guarantees.

Assessment

The truncated softmax is the most straightforward and preserves the theoretical framework. It requires choosing k and building a spatial index, but both are standard. The continuous map output is cleaner but requires rethinking how marginal constraints are enforced.

7. P7: Newton convergence basin [PARTIAL]

Problem

The warm-start strategy (P3) assumes the neural operator prediction $\hat{\psi}$ is close enough to ψ^* for Newton's method to converge in 2–3 steps. Newton has quadratic convergence but only within a basin of attraction. If $\hat{\psi}$ is far from ψ^* , Newton may diverge. Worst case: fall back to full iterative solve, so the amortized cost includes occasional expensive solves.

Partial solution

- **Damped Newton:** use line search or trust region to globalize Newton. Converges from any starting point (for the concave dual), but convergence is only linear outside the quadratic basin. Still faster than starting from scratch.
- **Monitor residual:** if $\|\nabla g(\hat{\psi})\|$ exceeds a threshold after the neural operator prediction, skip Newton and use a different solver (e.g., L-BFGS on the dual, or multiscale Sinkhorn).
- **Training with hard examples:** if certain input configurations consistently cause poor predictions, oversample them in the training set (active learning / hard example mining).

Open question

How often does the neural operator miss badly enough that 2–3 Newton steps are insufficient? This is an empirical question. If the answer is $< 1\%$ of resampling steps, the amortized cost is fine. If $> 10\%$, the warm-start strategy provides marginal benefit.

8. P8: Distribution shift between training and deployment [RESOLVED]

Problem

The neural operator is trained on $(\mathbf{w}, \mathbf{X})$ configurations from specific filtering runs. At deployment, the filter may encounter different configurations (different dynamical system, different observation noise, different initial conditions). The training distribution may not cover the deployment distribution.

Solution

The DAgger bootstrapping loop (Section 3.3 of the main document) addresses this for a *fixed* problem. For *new* problems, two options:

- **Fine-tuning:** run a short data-collection phase with Sinkhorn (or multiscale Sinkhorn) on the new problem, then fine-tune the neural operator. Fast if the pre-trained network is close.
- **Physics-informed regularization:** add the PINN loss (Monge–Ampère residual or entropic OT optimality conditions) during training. This provides a self-supervised signal that doesn’t depend on pre-computed solutions and encourages generalization beyond the training distribution.

9. P9: Curse of dimensionality at $d = 50$ –100 [OPEN]

Problem

At $d = 5$, the continuous density ρ_0 can be represented on a grid or approximated with moderate-dimensional random features. At $d = 50$, no generic representation works: grids are impossible (M^{50}), and random feature approximations of kernel mean embeddings degrade with dimension (the required number of features grows with d).

The neural operator must also approximate a map in $\mathbb{R}^{50} \rightarrow \mathbb{R}^{50}$. Universal approximation theorems guarantee existence but say nothing about the required network size, which may grow exponentially with d .

Possible directions

- **Exploit problem structure.** Most real filtering problems at $d = 50$ have low effective dimensionality: low-rank observation operators, sparse dynamics Jacobians, posterior concentration near manifolds. If the filtering density lives near a k -dimensional manifold with $k \ll d$, the neural operator only needs to learn a map in the effective k -dimensional space. The question is how to discover and exploit this automatically.
- **Learned compression.** Train an encoder $E : \mathbb{R}^d \rightarrow \mathbb{R}^k$ that maps the high-dimensional particle positions to a low-dimensional latent space, solve OT in latent space, decode back. This requires the encoder to preserve the OT structure (distances, measure), which is not guaranteed.
- **Factored transport.** If the state decomposes into weakly-coupled blocks (e.g., spatial locality), solve independent OT problems per block, similar to block particle filters. The neural operator handles each low-dimensional block, and blocks are coupled only through boundary conditions.

Assessment

This is the hardest open problem. The $d = 5$ proof of concept will not resolve it. The path from $d = 5$ to $d = 50$ requires at least one of the above structural assumptions, and which one applies depends on the specific target problem. This should be scoped as a separate research contribution.

10. P10: Encoding both source and target in the neural operator [OPEN]

Problem

The semi-discrete formulation assumes one continuous measure (source) and one discrete measure (target). The branch net encodes the source ρ_0 ; the target $(\mathbf{w}^1, \mathbf{y})$ enters as a parameter. But at $N = 10^6$, the target also has 10^6 atoms. How does the network process 10^6 target atoms?

If the output is $\psi^* \in \mathbb{R}^N$ (one dual variable per target atom), the output layer itself has $N = 10^6$ nodes. This makes the network N -dependent, defeating resolution invariance.

Possible directions

- **Symmetric compression:** compress both source and target into fixed-dimensional representations (random features, Fourier embeddings). The neural operator maps between two m -dimensional embeddings. Then recover ψ^* from the output embedding by evaluating at each target atom. Cost $O(Nm)$ for recovery.
- **Pointwise dual prediction:** instead of outputting all of $\psi^* \in \mathbb{R}^N$ at once, define $\psi_j^* = h_\theta(\mathbf{y}_j; \rho_0, \mu_1)$ — a function that predicts the j -th dual variable given the target position $\mathbf{y}_j$ and compressed representations of both measures. This is evaluated pointwise, $O(1)$ per target, $O(N)$ total. The network is N -independent. This is a natural extension of the DeepONet trunk idea.
- **Direct map (bypass duals):** output $T(\mathbf{x}) \in \mathbb{R}^d$ directly. The network never references individual target atoms. The map is implicitly defined by the training data. Loses explicit constraint satisfaction but avoids the N -dependent output problem entirely.

Assessment

The pointwise dual prediction is the cleanest: it preserves the OT dual structure while being N -independent. It requires that ψ_j^* varies smoothly with $\mathbf{y}_j$ (given fixed source and target measures), which is true when ρ_0 is smooth. This is probably the right design choice for production.

11. P11: Diffusion models are not suitable [RESOLVED]

Problem

Diffusion models have connections to OT (flow matching, Schrödinger bridges). Could they replace Sinkhorn?

Why not

- **Speed:** diffusion requires T sequential denoising steps (20–1000 network evaluations). Slower than a single forward pass. Defeats the purpose of acceleration.
- **Output structure:** diffusion generates samples, not transport plans or dual variables. Loses the coupling information needed for differentiable resampling.
- **Gradient variance:** differentiating through stochastic SDE integration requires reparameterization tricks, reintroducing the gradient variance that entropic OT was designed to eliminate.
- **Complexity:** the training pipeline (noise schedules, score matching, sampling strategies) adds engineering overhead for no clear benefit over supervised regression on Sinkhorn data.

12. P12: Is linear attention expressive enough? [PARTIAL]

Problem

Linear attention replaces $\text{softmax}(QK^\top)$ with $\varphi(Q)\varphi(K)^\top$, reducing cost from $O(N^2)$ to $O(N)$. But the OT problem involves pairwise costs $c(\mathbf{x}_i, \mathbf{y}_j)$. Can $O(N)$ computation capture $O(N^2)$ pairwise structure?

Current assessment

The entropic OT plan is approximately low-rank when ε is not too small: $\pi_{ij} \propto \exp(-c_{ij}/\varepsilon)$ concentrates mass on nearby pairs, and the kernel $\exp(-\|\mathbf{x}_i - \mathbf{y}_j\|^2/2\varepsilon)$ has effective rank that depends on ε and the data geometry, not on N . Linear attention with feature dimension p captures rank- p interactions. If p matches the effective rank of the OT plan, linear attention suffices.

For highly degenerate weights (near-permutation transport), the plan is sparse and full-rank. This is the hard regime. But the transport *map* (as opposed to the plan) may still be smooth even when the plan is sparse—the map sends each point to its nearest assigned target, and this assignment varies continuously in space when the source density is smooth.

Open question

Empirical comparison needed: linear attention vs. standard attention on the $d = 5$, small- N proof of concept. If accuracy gap is small ($< 5\%$ relative error on ψ^*), linear attention is validated. If large, investigate whether increasing the feature dimension p closes the gap.

13. P13: Choice of cost function and its interaction with the network [OPEN]

Problem

The standard OT cost is $c(\mathbf{x}, \mathbf{y}) = \frac{1}{2} \|\mathbf{x} - \mathbf{y}\|^2$. The entropic regularization parameter ε controls the softness of the transport plan. The neural operator must implicitly learn the relationship between c , ε , and ψ^* .

If ε is fixed, this is absorbed into the training data. If ε varies (e.g., annealed during filtering, or different per problem), it becomes an input to the network. Similarly, non-Euclidean costs (Riemannian manifolds, learned metrics) change the OT structure entirely.

Open questions

- Should ε be an input to the network, or should separate networks be trained for different ε values?
- If the cost function is non-standard (e.g., learned from data), can the neural operator generalize, or must it be retrained?
- The Monge–Ampère PINN loss assumes squared Euclidean cost (Brenier’s theorem). Non-Euclidean costs require different PDE formulations. How much of the framework survives a cost change?

14. P14: Validation and error quantification [OPEN]

Problem

How do you know the neural operator is working correctly in production? At $N = 10^6$, you cannot run Sinkhorn to check. The Newton residual $\|\nabla g(\hat{\psi})\|$ gives a local optimality measure, but computing ∇g itself requires evaluating integrals over Laguerre cells or the softmax assignment—potentially expensive.

Possible directions

- **Marginal check:** verify that the transport plan approximately satisfies the marginal constraints. For the soft assignment, check $\sum_j \pi_j(\mathbf{x}_i) \approx \rho_0(\mathbf{x}_i)$ and $\int \pi_j(\mathbf{x}) d\mathbf{x} \approx w_j^1$ on a subsample. Cost $O(Nk)$ with k nearest neighbors.
- **Dual gap:** compute the primal and dual objectives on a subsample and check that the gap is small. Provides a certificate of near-optimality.
- **Filter-level validation:** monitor the ESS and state estimation error of the particle filter. If these degrade relative to a baseline (small- N Sinkhorn filter), the resampling is failing.

Assessment

The filter-level validation is the most practical: you care about filtering accuracy, not OT optimality per se. A slight suboptimality in the transport plan may be harmless if the filter still tracks the state. Define acceptance criteria in terms of filtering metrics, not OT metrics.

Summary Table

#	Problem	Status	Blocking?
P1	Sinkhorn infeasible at $N = 10^6$	[RESOLVED]	Yes (forces map formulation)
P2	Transport map not differentiable	[RESOLVED]	Yes (entropic regularization)
P3	Neural operator gradient bias	[RESOLVED]	Yes (Newton warm start)
P4	Data validity under inference	[RESOLVED]	No
P5	Representing ρ_0 at scale	[PARTIAL]	Yes for production
P6	Soft assignment $O(N^2)$ cost	[OPEN]	Yes for production
P7	Newton convergence basin	[PARTIAL]	No (graceful fallback)
P8	Distribution shift	[RESOLVED]	No (DAgger + fine-tuning)
P9	Curse of dimensionality $d = 50+$	[OPEN]	Not for $d = 5$ POC
P10	Encoding both measures at scale	[OPEN]	Yes for production
P11	Diffusion models unsuitable	[RESOLVED]	No (eliminated)
P12	Linear attention expressiveness	[PARTIAL]	No (empirical test)
P13	Cost function interaction	[OPEN]	Not for POC
P14	Validation at scale	[OPEN]	Yes for production

Critical path for proof of concept ($d = 5$, moderate N): P1 $\rightarrow$ P2 $\rightarrow$ P3 $\rightarrow$ P12.

Critical path for production ($N = 10^6$): P5 $\rightarrow$ P6 $\rightarrow$ P10 $\rightarrow$ P14.

Critical path for high dimension ($d = 50+$): P9 (requires separate research contribution).